REPORT · 2026年8月

造像类文物三维建模自动化系统 register代码说明书

M2 配准 · FPFH+RANSAC 粗配准 + ICP 精配准

文保数字化研发组
2026年8月

目 录

一、概述 3

二、模块划分 4

三、依赖关系与数据流 5

3.1 依赖方向 5

3.2 两段式配准数据流 5

四、模块详细设计 6

4.1 feature.py — FPFH 特征提取 6

4.2 ransac.py — RANSAC 全局粗配准 6

4.3 icp.py — point-to-plane ICP 精配准 6

4.4 evaluate.py — 误差评估 6

4.5 __init__.py — run 两段式流水线 7

五、接口契约与错误码 8

六、测试与验证 9

一、概述

本文档是《造像类文物三维建模自动化系统（路线 E）》**register 子包（M2 配准，L2 业务层 / S2 阶段）**的代码说明书。register 负责把两片（异源 / 多站）点云对齐到统一坐标系，采用**FPFH+RANSAC 全局粗配准 + point-to-plane ICP 精配准**两段式策略，并输出配准残差 RMS 与内点占比作为质量指标。本说明书严格遵循《软件开发项目设计书》第 15、33 章（M2 配准）、《管线骨架构建分析》第 4~5 章（模块划分与依赖约束）及《开发指南（教学版）》第 16 章的接口约定。

> 设计要点：两段式配准——先用 FPFH+RANSAC 在**不依赖初值**的前提下求全局粗变换，再用 ICP 在粗变换基础上做**局部精化**。粗配准解决「差得远」，精配准解决「对得准」，二者缺一不可。

二、模块划分

register 子包由 4 个业务模块与 1 个两段式流水线构成，目录结构与职责如下：

```
src/h3d/register/
+-- __init__.py     # 包导出 + run(source, target, cfg) 两段式配准流水线
+-- feature.py     # FPFH 特征提取（自动补齐法线）
+-- ransac.py      # RANSAC 全局粗配准
+-- icp.py         # point-to-plane ICP 精配准
+-- evaluate.py    # RMSE / fitness 配准误差评估
```

模块文件	核心职责	对外关键符号
feature.py	FPFH 特征提取（法线自动补齐）	compute_fpfh, ensure_normals
ransac.py	RANSAC 全局粗配准	global_registration
icp.py	point-to-plane ICP 精配准	icp_refine
evaluate.py	RMSE / fitness 误差评估	fitness, inlier_rmse, correspondence_count, evaluate_registration
__init__.py	包导出 + 两段式配准流水线	run(source, target, cfg)

四个模块构成「特征 → 粗配准 → 精配准 → 评估」的配准链：**feature** 提供匹配描述子、**ransac** 求全局初值、**icp** 做局部精化、**evaluate** 量化结果。四者通过 **run()** 串接，参数全部取自 **RegisterConfig**。

三、依赖关系与数据流

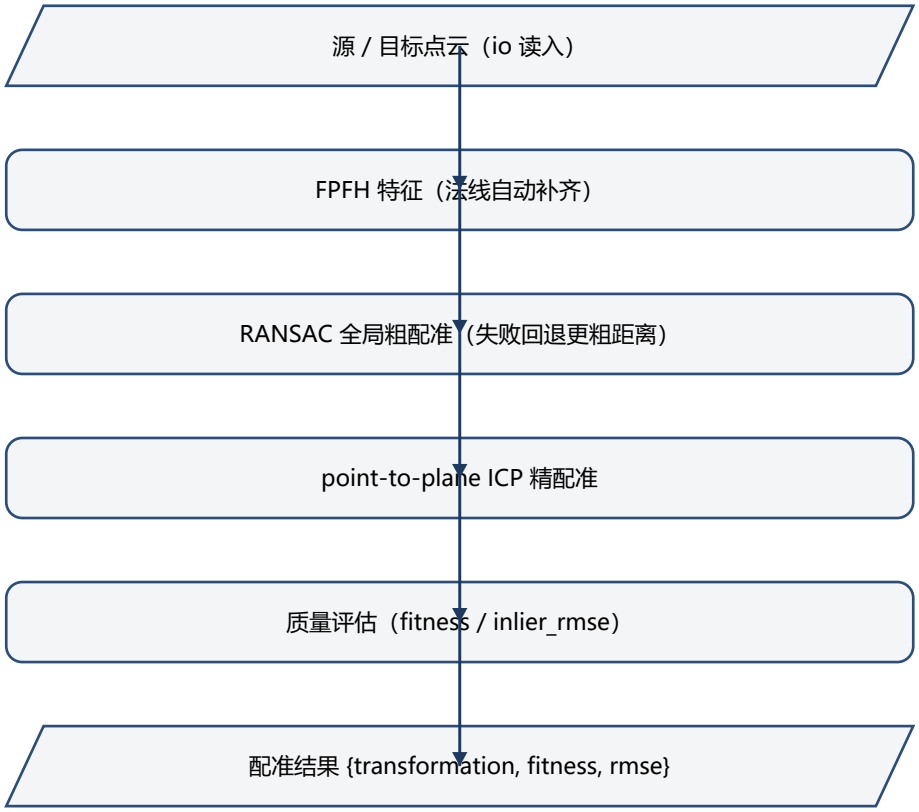
3.1 依赖方向

依据《管线骨架构建分析》5.2，register 属 L2 业务层，依赖 core 底座与 io（数据读入），不反向。输入点云经 io / preprocess 层读入并已带法线：

```
package-level deps (A -> B means A imports B):
  register  -->  core.exceptions (InputDataError, RegistrationError)
  register  -->  core.config    (RegisterConfig)
  register  -->  core.logging   (get_logger)
  register  -->  Open3D.pipelines.registration (几何配准，惰性导入)
  register  -->  (数据经 io/preprocess 读入，不直接读写文件)
  core / io / preprocess  -->  (NONE; 不反向依赖 register)
```

3.2 两段式配准数据流

run(source, target, cfg) 两段式配准的完整流转（自顶向下，精配准后 fitness 低于阈值抛 RegistrationError E200）：



run() 两段式配准数据流：特征 → 粗配准 → 精配准 → 评估（fitness < 0.2 抛 E200）

四、模块详细设计

4.1 feature.py — FPFH 特征提取

FPFH（Fast Point Feature Histogram）以法线为基础，编码点云局部邻域的几何分布，用于 RANSAC 的特征匹配。输入点云缺法线时自动补齐（正常由 preprocess 层提供，此处兜底）。FPFH 对初始位姿不敏感，是全局配准「无需初值」的关键。

函数	签名	职责
compute_fpfh	(pcd, radius, max_nn=100) -> Feature	计算 FPFH 特征（33 维直方图）
ensure_normals	(pcd, radius, max_nn=30) -> PointCloud	法线缺失时自动补齐（原地修改）

4.2 ransac.py — RANSAC 全局粗配准

RANSAC 通过随机抽样一致，从含噪的 FPFH 特征对应中估计稳健刚体变换。使用 mutual_filter 与边长 / 距离双重校验器过滤错误对应，为 ICP 提供良好初值。

```
def global_registration(source, target, source_fpfh, target_fpfh,
                        max_correspondence_distance, ransac_n=4,
                        max_iteration=100000):
    return registration_ransac_based_on_feature_matching(
        source, target, source_fpfh, target_fpfh,
        mutual_filter=True,
        max_correspondence_distance=max_correspondence_distance,
        estimation_method=TransformationEstimationPointToPoint(False),
        ransac_n=ransac_n,
        checkers=[CorrespondenceCheckerBasedOnEdgeLength(0.9),
                  CorrespondenceCheckerBasedOnDistance(...)],
        criteria=RANSACConvergenceCriteria(max_iteration, 0.999))
```

4.3 icp.py — point-to-plane ICP 精配准

ICP 在粗配准初值上迭代优化。point-to-plane 变体最小化点到目标面切平面的距离平方和，比 point-to-point 更贴合曲面、收敛更快、精度更高；目标缺法线时自动退化为 point-to-point。

```
def icp_refine(source, target, init, max_correspondence_distance,
               max_iteration=2000, point_to_plane=True):
    estimation = (TransformationEstimationPointToPlane()
                  if point_to_plane
                  else TransformationEstimationPointToPoint())
    return registration_icp(
        source, target, max_correspondence_distance, init, estimation,
        ICPConvergenceCriteria(max_iteration=max_iteration,
                               relative_fitness=1e-6, relative_rmse=1e-6))
```

4.4 evaluate.py — 误差评估

从 RegistrationResult 提取质量指标：fitness（内点占比）、inlier_rmse（内点均方根误差）、对应点数量。配准误差是后续质检（M5）与 WW/T 0117 精度对标的关键输入。

函数	含义
fitness(result)	内点占比（0~1，越高越好）

函数	含义
inlier_rmse(result)	内点均方根误差（越低越好）
correspondence_count(result)	对应点数量
evaluate_registration(result)	汇总为结构化 dict

4.5 __init__.py — run 两段式流水线

run(source, target, cfg) 是 register 的对外入口：法线补齐 + FPFH → RANSAC 粗配准（fitness 低时回退更粗距离重试）→ ICP 精配准 → 质量判定。精配准后 fitness 仍低于阈值（0.2）即抛 E200（重叠不足）。

```
def run(source, target, cfg: RegisterConfig) -> dict:
    f_src = compute_fpfh(source, cfg.fpfh_radius)
    f_tgt = compute_fpfh(target, cfg.fpfh_radius)
    ransac = _ransac_with_fallback(source, target, f_src, f_tgt,
                                   cfg.ransac_distance)
    icp = icp_refine(source, target, ransac.transformation,
                     cfg.icp_distance, cfg.max_iter)
    if icp.fitness < 0.2:              # 重叠不足
        raise RegistrationError("配准失败（重叠不足）")
    return {**evaluate_registration(icp),
            "transformation": icp.transformation}
```

配置项	默认值	用途
register.fpfh_radius	0.02	FPFH 特征半径（米）
register.ransac_distance	0.05	RANSAC 最大对应距离（米）
register.icp_distance	0.02	ICP 最大对应距离（米）
register.max_iter	2000	ICP 最大迭代次数

五、接口契约与错误码

register 对外暴露纯函数式接口（PointCloud 进、结果 dict 出），错误码与设计书 13.3 严格对齐：输入空点云抛 InputDataError(E100)，重叠不足 / 配准失败抛 RegistrationError(E200)。

错误码	异常类	触发场景	处理策略
E100	InputDataError	输入点云为空 / FPFH 特征缺失 / 初始变换缺失	标记跳过，记入 manifest（可恢复）
E200	RegistrationError	重叠不足（精配准后 fitness < 0.2）	建议补扫或人工约束，记入 manifest（可恢复）

六、测试与验证

register 配套单元测试位于 **tests/test_register/**，用带凹凸的非对称球面（可匹配的 FPFH 特征）验证已知变换恢复、失败隔离与错误码，共 11 项，全部通过。

测试文件	覆盖点
test_feature.py	FPFH 33 维特征、法线自动补齐、幂等、空输入 → E100
test_evaluate.py	fitness / inlier_rmse / 对应点数量提取
test_pipeline.py	已知变换恢复（平移误差 < 5cm）、精配准优于粗配准、空输入 → E100、重叠不足 → E200

```
$ python -m pytest tests/test_register -q
...........                              [100%]
11 passed in 0.68s

$ python -m pytest -q   # 全量（core + io + preprocess + register）
.....................................................................    [100%]
76 passed in 4.45s
```

运行方式：在 .venv_lab 虚拟环境执行 pytest；或 python -c "from h3d.io import load_point_cloud; from h3d.register import run; from h3d.core import load" 冒烟验证。核心依赖全部 MIT/BSD。